Breaking the Composition Cliff: Mutation-Guided Specification Synthesis for Multi-Function Programs

— Abstract —

Ishraq Tanvir
FOF Business Point
ishraq.tanvir@fofbd.com

Abstract

Large language models (LLMs) can generate code, but synthesizing formal specifications for this code remains challenging. While previous approaches report high verification rates on individual functions, real-world programs are composed of modular interprocedural call graphs. We evaluate six frontier language models on synthesizing formal Dafny specifications for 300 multi-function programs with call-graphs of sizes 2–5 (DAFNYCOMP-300), revealing two key shortcomings. First, we find that models struggle with compositional specifications: while models reach 91.7%–100% specification completeness on individual functions with repair, zero-shot specification success collapses from up to 95.0% on single functions down to 14.0%–41.3% on multi-function call graphs. Second, the traditional repair loop, which relies on compiler error messages, leads models to learn vacuous specifications, e.g., trivial inequalities such as `ensures res >= 0`. While these specifications appear logically sound, they are overly weak and cannot reject incorrect implementations. To address this, we present MUTATION-CEGIS, an approach that uses AST code mutants to test specifications for vacuity. When a candidate specification mistakenly admits a buggy mutant, we extract the code change as a concrete counterexample for the model to refine its specification. On single-function benchmarks (CLOVERBENCH-60), MUTATION-CEGIS boosts specification completeness to 91.7%–100.0% while reducing repair turns by up to 36.8% and saving up to 24.9% of total tokens. On compositional call graphs (DAFNYCOMP-300), MUTATION-CEGIS systematically detects vacuous bounding contracts across model families and, when synthesis succeeds, drives models toward tighter first-order relational specifications ($\forall, \exists$). Finally, we find that for deeper call-graphs, exact specification synthesis results in SMT-solver timeouts, demonstrating that future verification agents will benefit from synthesizing auxiliary lemmas in addition to specifications.

Keywords: Formal Verification, Program Synthesis, CEGIS, Mutation Testing, Deductive Verification, Code LLMs, AI Agents, Agent Verification, Dafny, SMT Solvers, Z3, Hoare Logic, Invariant Synthesis, Software Reliability.

TL;DR: We uncover the Composition Cliff in LLM formal verification and present MUTATION-CEGIS using AST code mutants to eliminate vacuous contracts and synthesize hardened first-order specifications across multi-function programs.

Code & Benchmarks: `https://github.com/devishraq/composition-cliff`

Workshop on Who Verifies the Agents? Toward Reliable Agent Development, NeurIPS 2026.